

Fundamentals of Natural Language Processing

Chapter 1 — The NLP Landscape

Aymen Ben Brik

Chapter 1

The NLP Landscape

This opening chapter situates Natural Language Processing as a field: what problem it tries to solve, what makes that problem fundamentally hard, what canonical tasks define the discipline, and how its methods have evolved from hand-written rules to billion-parameter language models. Throughout, the goal is to develop a precise mental map you can return to as more advanced techniques are introduced.

1.1 What is NLP? Definition, scope, and core challenges

1.1.1 Motivation

Computers are excellent with *structured* data: spreadsheets, SQL tables, signed numerical vectors. They struggle with *unstructured* human language — the messy, fluid, ambiguous medium humans use to communicate. Yet a vast majority of the world’s data exists in textual form: emails, chat messages, news articles, scientific papers, social-media posts, customer reviews, medical records, contracts. Building machines that can read, summarize, translate, search, and converse in natural language unlocks nearly every modern AI product, from search engines and recommendation systems to clinical decision-support tools and large language model assistants. *Natural Language Processing* (NLP) is the field that develops and studies the algorithmic foundations of this capability.

1.1.2 Approach by problem: the headline that everyone misreads

Problem. Consider the (real) news headline:

“Stolen painting found by tree.”

List every grammatically valid reading. How many require world knowledge to discard? How could an algorithm resolve the intended one?

Solution sketch. The headline admits at least two parses:

1. “Stolen painting [found by [tree]]” — a tree found the painting.
2. “Stolen painting [found [by tree]]” — the painting was found beside a tree.

Reading 1 is grammatically valid but pragmatically absurd: trees do not perform searches. A native speaker rules it out using world knowledge (trees are inanimate). A computer that does *not* have such knowledge cannot rule it out without an explicit mechanism — whether a hand-coded rule, statistical prior, or learned representation. Multiplying this difficulty across millions of sentences, in dozens of languages, is the central challenge NLP must address.

1.1.3 Definition and scope

Definition 1.1.1 (Natural Language Processing). **Natural Language Processing** (NLP) is the subfield of artificial intelligence concerned with enabling computer systems to process, understand, and generate human language. It sits at the intersection of *linguistics* (the scientific study of language), *computer science* (algorithms and data structures) and *machine learning* (data-driven model building).

It is convenient to organise the field along three complementary axes:

- **Natural Language Understanding (NLU)** — extracting meaning, structure, sentiment, intent, or entities from text. *Inputs* are texts, *outputs* are typically structured representations (labels, parse trees, knowledge-graph triples).
- **Natural Language Generation (NLG)** — producing fluent and coherent natural-language outputs from structured inputs (database rows, dialogue states, summaries of long documents).
- **Interaction** — dialogue systems and conversational agents, where understanding and generation operate in a closed loop with a user.

1.1.4 Why natural language is hard

Three orthogonal sources of difficulty pervade every NLP system.

1. Ambiguity

A single linguistic expression can correspond to multiple meanings or structures. Ambiguity arises at every level of analysis.

- **Lexical ambiguity** (polysemy or homonymy). A single word denotes several distinct concepts.
 - *bank*: financial institution VS. edge of a river.
 - *date*: calendar day VS. romantic appointment VS. fruit.
- **Syntactic ambiguity**. A single string can be parsed by several distinct grammatical structures.
 - “*I saw the man with the telescope.*” — I used the telescope, or the man had a telescope?
- **Semantic ambiguity**. A single syntactic structure admits several meanings.
 - “*The chicken is ready to eat.*” — the chicken will eat, or will be eaten?

- **Pragmatic ambiguity.** The intended communicative function is underdetermined by the literal content.
 - “*Can you pass the salt?*” is a polite request, not a yes/no inquiry about the listener’s capabilities.

2. Context dependence

The meaning of an expression cannot be computed from its surface form alone — it depends on its surrounding linguistic context and on world knowledge.

- “*Apple released a new product.*” (the company) vs. “*Apple is healthy.*” (the fruit). The same string, two referents.
- Pronoun resolution: “*John told Bill that he had failed the exam.*” — who failed? Either reading is grammatical; only context decides.
- Sarcasm and irony: “*Oh great, another delay.*” may carry the opposite of its literal meaning.

3. Variability

Many distinct surface forms encode the same meaning, and natural language permits substantial creativity.

- **Synonymy and paraphrasing:** “*happy*”, “*joyful*”, “*content*”, “*elated*” are near-synonyms with different connotations.
- **Spelling and orthographic variation:** “*colour*” / “*color*”, typos, leetspeak, abbreviations.
- **Cross-domain shifts:** medical, legal, social-media, and academic text differ in vocabulary, syntax, and conventions.
- **Multilingualism and code-switching:** “*J’ai*” un meeting demain — many speakers freely mix languages.

Remark 1.1.1. These three challenges combine multiplicatively. An algorithm that handles ambiguity but not context will fail on the simplest pronoun resolution; one that handles context but not variability will not generalize from training to deployment data.

1.1.5 Worked examples

Example 1 (Lexical ambiguity — Easy). Identify the polysemous word in the following sentence and give at least two distinct readings:

She left her glasses on the table.

Solution. The polysemous word is *glasses*, which may refer to:

- eyewear (a pair of corrective spectacles), or
- drinking vessels (e.g. wine glasses).

World knowledge — e.g. that the speaker is myopic, or that the table held a meal — is needed to disambiguate.

Example 2 (Syntactic ambiguity — Medium). Resolve the ambiguity of “*He hit the boy with the bat*” under the following two contexts:

Context A: The previous sentence was “*He grabbed a wooden bat from the dugout.*”

Context B: The previous sentence was “*He met the boy with the bat at the zoo.*”

Solution. In context A, “*with the bat*” attaches to the verb *hit* (instrumental adjunct): the bat is the weapon. In context B, “*with the bat*” attaches to the noun *boy* (modifier): the boy carried a bat (the animal). Same surface form, two parse trees, two meanings; the intended one is selected only via context.

Example 3 (Pragmatics over literal content — Hard). The utterance “*It is cold in here.*” can play several pragmatic roles. List three plausible interpretations and explain when each applies.

Solution.

1. **Literal assertion:** a factual statement about the temperature, expecting acknowledgement.
2. **Indirect request:** an implicit suggestion to close a window, turn up heating, or hand the speaker a sweater (when the speaker has the social standing to make a request).
3. **Sarcasm:** in a room that is unmistakably warm, the same string may serve as ironic commentary on, e.g., an awkward silence.

Selecting the correct reading requires modelling the speaker, the listener, the physical situation, and the broader cultural conventions of indirect speech — topics studied under the name of *pragmatics*.

1.1.6 Python snippet: probing word senses

The Princeton WordNet lexical database, accessible through NLTK, allows us to enumerate distinct senses of a polysemous word.

```
1 import nltk
2 nltk.download('wordnet', quiet=True)
3 from nltk.corpus import wordnet as wn
4
5 def show_senses(word):
6     senses = wn.synsets(word)
7     print(f"'{word}' has {len(senses)} sense(s) in WordNet:")
8     for syn in senses[:6]:
9         print(f"    {syn.name():<20s} {syn.definition()}")
10
11 show_senses("bank")
12 show_senses("date")
13 show_senses("light")
```

Running this snippet illustrates that even commonplace words carry many distinct senses — the raw input to any system that aspires to “understand” text.

1.1.7 Exercises

1. Find three polysemous English words used in everyday speech, and list at least two distinct senses of each. For each pair of senses, propose a sentence in which only one of them is felicitous.
2. For the sentence “*The duchess was entertaining last night,*” identify two grammatical readings and explain how the ambiguity is structured (lexical or syntactic).
3. Find a real-world headline (e.g. from the Wikipedia article on “Crash blossom” or from a news aggregator) that exhibits at least one form of ambiguity. Classify the ambiguity (lexical / syntactic / semantic / pragmatic) and propose a context that resolves it.
4. Discuss: how might *world knowledge* help a system disambiguate “*I deposited the money in the bank*”? What form would such knowledge take, and where might it be obtained at scale?
5. (Synthesis.) Suppose you must build a customer-service chatbot for a Tunisian e-commerce site that receives messages mixing French, Arabic and English. Identify, for each of the three challenges (*ambiguity*, *context*, *variability*), one concrete obstacle your system will face that an English-only system would not.

1.2 Major NLP tasks and applications

1.2.1 Motivation

NLP is not a single problem; it is a constellation of problems, each with its own *input format*, *output structure*, *evaluation metric*, and *characteristic methods*. A sentiment classifier and a machine-translation system both consume text, but they belong to different computational categories and therefore demand different architectures and training data. Practitioners must develop a clear taxonomy of NLP tasks for three practical reasons:

1. to *frame* a real-world business problem as the right computational task;
2. to *select* the appropriate model family (classification vs. sequence labelling vs. generation);
3. to *evaluate* models with metrics that match the task — accuracy is meaningful for classification but disastrous as a sole metric for translation.

1.2.2 Approach by problem: one dataset, many tasks

Problem. You receive a corpus of 50,000 customer reviews of a hotel chain. Each review is a paragraph of free text, sometimes with a star rating, sometimes not. Your manager asks: “What can we do with this data?” List at least five distinct NLP problems you could solve from *the same* raw input.

Solution sketch. The same corpus can power multiple downstream tasks:

1. **Sentiment analysis** — classify each review as positive, negative, or neutral.
2. **Topic classification** — assign reviews to themes (food, cleanliness, location, staff).

3. **Aspect-based sentiment** — extract sentiment per aspect (“rooms: positive, breakfast: negative”).
4. **Named entity recognition** — extract place names, dates, hotel branches mentioned.
5. **Summarization** — produce a one-sentence digest of each review.
6. **Translation** — translate Arabic and French reviews into English for cross-lingual analytics.
7. **Question answering** — support a query interface (“What do guests say about the pool?”).

Each item belongs to a different computational category, requires different annotated data, and is evaluated with different metrics. The remainder of this section gives a structured tour of these categories.

1.2.3 A taxonomy by input–output structure

We organise NLP tasks by what the model *consumes* and *produces*.

Category	Input	Output
Text classification	text	one (or several) discrete labels
Sequence labelling	text	one label per token
Sequence-to-sequence	text	a different text
Question answering	question + context	answer (span or text)
Open-ended generation	prompt	arbitrary text

The categories form a rough *spectrum of output complexity*, from a single integer (classification) to a free-form sequence (generation). Each step adds expressive power and evaluation difficulty.

1.2.4 Text classification

Definition 1.2.1 (Text classification). Given an input text x drawn from a domain $\mathcal{X}$ and a finite label set $\mathcal{Y} = \{y_1, \dots, y_K\}$, learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ (single-label) or $f : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{Y})$ (multi-label) that assigns labels to texts.

Canonical applications.

- **Sentiment analysis** — $\mathcal{Y} = \{\text{positive, negative, neutral}\}$. Powers product-review analytics, social-media monitoring, financial-sentiment trading signals.
- **Spam detection** — $\mathcal{Y} = \{\text{spam, ham}\}$. The earliest commercially deployed NLP application at scale.
- **Topic classification** — assign a document to one of several thematic categories (politics, sports, technology, ...).
- **Intent detection** — in chatbots, classify a user utterance into pre-defined intents (BOOK-FLIGHT, CANCEL-ORDER, ASK-STATUS).
- **Toxicity / hate-speech detection** — moderate user-generated content on social platforms.

Standard metrics. Accuracy, Precision, Recall, F_1 (Section ?? will define them precisely).

1.2.5 Sequence labelling

Definition 1.2.2 (Sequence labelling). Given a sequence of tokens $x = (x_1, x_2, \dots, x_n)$, predict a sequence of labels $y = (y_1, y_2, \dots, y_n)$ of the same length, where each y_i belongs to a finite label set $\mathcal{Y}$.

The output dimension grows with the input, in contrast to classification.

Canonical applications.

- **Part-of-speech (POS) tagging** — label every token with its grammatical role: *noun, verb, adjective, ...*

“Apple/NOUN released/VERB a/DET new/ADJ phone/NOUN.”

- **Named Entity Recognition (NER)** — label tokens that participate in named entities (persons, organizations, locations, dates) with their type and span.

“Apple/B-ORG was/O founded/O by/O Steve/B-PER Jobs/I-PER in/O Cupertino/B-LOC.”

The B-/I-/O scheme distinguishes *Begin* of an entity, *Inside*, and *Outside*.

- **Chunking and shallow parsing** — group tokens into noun phrases, verb phrases.
- **Slot filling** — in dialogue systems, identify the DATE, ORIGIN, DESTINATION slots in user utterances.

High-impact applications. Pharmacovigilance (drug-name extraction from medical records), legal-document analysis (clause boundaries), bioinformatics (gene and protein-name recognition), search-engine query understanding.

1.2.6 Sequence-to-sequence

Definition 1.2.3 (Sequence-to-sequence task). Given a source sequence $x = (x_1, \dots, x_n)$, produce a target sequence $y = (y_1, \dots, y_m)$ where m may differ from n and the target vocabulary may differ from the source.

This category is the most general; it subsumes any task where the output is itself a piece of text.

Canonical applications.

- **Machine translation** — map a sentence in source language L_1 to its translation in target language L_2 . Output length, word order, and idioms differ from the source.
- **Abstractive summarization** — produce a short summary that need not reuse the exact wording of the input. Contrast with *extractive* summarization, which selects sentences from the input verbatim.

- **Paraphrasing and style transfer** — rewrite a text with the same meaning but different surface form (e.g. formal to informal).
- **Grammar correction** — map an error-laden sentence to its corrected version.

Standard metrics. BLEU (translation), ROUGE (summarization). Both are studied formally in Section ??.

1.2.7 Question answering

Definition 1.2.4 (Question answering). Given a question q and (optionally) a context document or knowledge base c , produce an answer a to the question.

QA is conceptually a special case of sequence-to-sequence but is distinguished by the structure of the input (question + context) and by several distinct sub-paradigms:

- **Extractive QA.** The answer is a contiguous span of the context document. The model’s job is to predict the start and end positions. Typical of reading-comprehension benchmarks (SQuAD).
- **Closed-book QA.** No context is provided; the model must rely on knowledge stored in its parameters.
- **Open-domain QA.** The model first *retrieves* relevant documents from a large corpus, then reads them to extract the answer. This is the foundation of the *retrieval-augmented generation* (RAG) pattern.
- **Generative QA.** The answer is generated freely, possibly synthesising information from the context.

Industrial deployments. Customer-service chatbots (intent+slot extraction), Web search (featured snippets), enterprise document search.

1.2.8 Open-ended generation

Definition 1.2.5 (Open-ended generation). Given a prompt x , produce a continuation y that is fluent, coherent, and consistent with x but otherwise underdetermined.

This is the regime of large language models. The task is intentionally under-specified: many distinct continuations may be acceptable. Evaluation is therefore especially challenging.

Applications. Creative writing assistance, code generation (autocomplete and full-function synthesis), dialogue agents, automatic e-mail drafting, automated report generation from structured data.

1.2.9 Worked examples

Example 1 (Task framing — Easy). You are given the following row from a dataset of medical encounter notes:

Patient (45 y, F) reports persistent headache and was prescribed Paracetamol 500 mg twice a day.

Identify two distinct NLP tasks that operate on this single sentence, naming the appropriate category for each.

Solution.

- **Named Entity Recognition** (sequence labelling): extract [Drug: Paracetamol], [Dose: 500 mg], [Frequency: twice a day], [Symptom: headache].
- **Text classification**: classify the note’s clinical specialty (e.g. *neurology*).

Example 2 (Choosing the right output structure — Medium). You wish to build a system that, given a research-paper abstract, produces “the three most important takeaways”. Should you frame this as a sequence-to-sequence task or as a classification task? Justify.

Solution. A classification task forces the output into a finite, pre-defined set of labels — but the “takeaways” are free-form sentences whose content depends on each abstract. The natural framing is therefore *abstractive summarization* (sequence-to-sequence), with an additional length constraint that forces the output to consist of three short sentences. A pre-defined-label classification approach would be tractable only if you replaced “the three most important takeaways” by “one of K predefined research-area tags,” which is a substantively different problem.

Example 3 (Decomposing a complex query — Hard). A user asks the chatbot of a Tunisian airline: “*I bought a ticket to Paris last week, but my passport just expired. What should I do?*” Decompose this single utterance into the NLP sub-tasks the system must execute to respond usefully.

Solution.

1. **Intent detection** (classification): the intent is approximately HELP-WITH-DOCUMENT-ISSUE or CHANGE-BOOKING.
2. **Slot filling / NER** (sequence labelling): extract [Destination: Paris], [BookingDate: last week], [DocumentIssue: expired passport].
3. **Knowledge retrieval** (open-domain QA): fetch the airline’s policy on travel-document issues.
4. **Generation**: synthesise an empathetic, accurate, and fluent response that explains the available options (refund, ticket transfer, document renewal).

A modern dialogue system performs these steps either as separate components in a pipeline, or jointly inside a single large language model that is prompted with the user message and policy documents.

1.2.10 Python snippet: spaCy pipeline

The library spaCy ships pre-trained models that perform several of the tasks above in a single forward pass.

```
1 # pip install spacy && python -m spacy download en_core_web_sm
2 import spacy
3 nlp = spacy.load("en_core_web_sm")
4 text = ("Apple was founded by Steve Jobs in Cupertino in 1976. "
5         "Today, the company is worth over $3 trillion.")
6 doc = nlp(text)
7
8 print("Tokens with POS tags:")
9 for token in doc[:10]:
10     print(f"_{token.text:<12s}_{token.pos_:<6s}_{token.dep_}")
11
12 print("\nNamed entities:")
13 for ent in doc.ents:
14     print(f"_{ent.text:<20s}_{ent.label_}")
```

A single call returns tokenisation, POS tags, dependency parse, and named entities — a concrete instantiation of the sequence-labelling and classification ideas of this section.

1.2.11 Exercises

1. For each of the following, identify the most appropriate NLP task category (classification, sequence labelling, sequence-to-sequence, QA, generation):
 - (a) Detecting whether an SMS is fraudulent.
 - (b) Translating a Tunisian Arabic Facebook post into French.
 - (c) Extracting the names of all medications mentioned in a pharmacist's note.
 - (d) Producing a one-paragraph summary of a court ruling.
 - (e) Determining the writer's age range from a blog post.
2. For *aspect-based sentiment analysis* (e.g. "the food was great but the service was slow"), argue that the natural framing combines two distinct task categories. Which two?
3. Choose one industry (healthcare, finance, legal, education) and list three concrete NLP applications it uses. For each, identify the input, the output, and the dominant challenge from Section 1.1.
4. Discuss: should a search-engine query understanding system be framed as classification, sequence labelling, or generation? Defend your choice and identify trade-offs.
5. (Open-ended.) Sketch a pipeline that, given a Tunisian e-commerce review written in code-mixed French–Arabic, produces (a) an overall sentiment, (b) a list of mentioned product aspects, and (c) an English summary. Identify each NLP task involved and the order of execution.